

Jogo de Reflexo em FPGA

Spartan-3 XC3S200 · Bionexus TX-LED R27

Expande Tech

Felipe · Maicon · Paulo Henrique · Willian Colleti

Disciplina: Linguagem de Descrição de Hardware · Junho de 2026

Desafio Recebido

O que foi proposto pelo professor

Implementar projeto digital em VHDL para FPGA, com gravação permanente via USB e comunicação de resultados ao PC.

Entregáveis obrigatórios:

- Código VHDL base (Pisca LED 1 Hz)
- Projeto principal: Jogo de Reflexo
- Documento técnico da placa
- Slides de apresentação

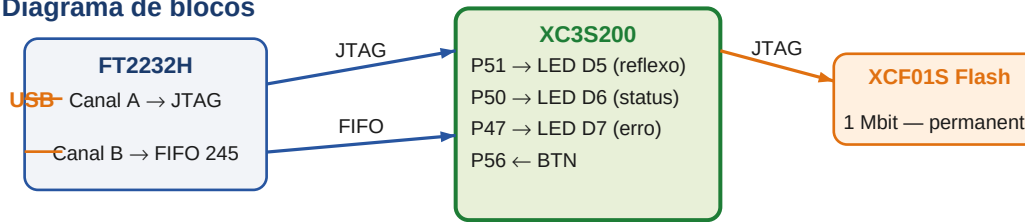
Hardware fornecido:

Campo	Valor
Placa	Bionexus TX-LED R27
FPGA	Xilinx XC3S200-4TQG144
Clock	40 MHz
Alimentação	16 V DC / 500 mA
USB	FT2232H dual-channel
Flash	XCF01S 1 Mbit
Gravação	xc3sprog / OpenOCD via USB

A Placa: Bionexus TX-LED R27

Identificação e componentes principais

Diagrama de blocos



Especificações

Campo	Valor
Placa	Bionexus TX-LED R27
FPGA	XC3S200-4TQG144
Clock	40 MHz (P53)
Alimentação	16 V DC / 500 mA
LEDs	D5(P51) · D6(P50) · D7(P47)
Botão	P56 (BLE_STS) — pull-down BLE
USB	FT2232H 0403:70b1
Flash	XCF01S — 1 Mbit

Requisitos do Jogo de Reflexo

O que o sistema deve fazer

O jogador pressiona um botão e o sistema mede quanto tempo levou para reagir ao LED.

Fluxo de uma partida:



ID	Requisito	OK
RF-01	D6 aceso em IDLE (aguardando)	✓
RF-02	LED D5 acende após intervalo aleatório 1–5 s	✓
RF-03	Mede tempo de reação em ms (resolução 1 ms)	✓
RF-04	D7 acende ao pressionar cedo (EARLY)	✓
RF-05	D7 acende em TIMEOUT (9.999 ms sem reação)	✓
RF-06	Envia resultado ao PC via USB FIFO	✓
RF-07	Firmware permanente — carrega sem PC	✓

Pinagem Utilizada

Mapeamento sinal VHDL → pino físico → periférico

Sinais do jogo

Sinal VHDL	Pino	Dir.	Função
clk	P53	Entrada	Clock 40 MHz
btn	P56	Entrada	Botão (pull-down BLE)
led_reflexo	P51	Saída	LED D5 — hora de reagir
led_status	P50	Saída	LED D6 — IDLE / OK
led_erro	P47	Saída	LED D7 — EARLY/TIMEOUT

Interface FIFO 245 — Canal B FT2232H (Bank 6)

Sinal	Pinos	Dir.	Função
usb_d[0..7]	P31/P30/P27/P28/P32/P33/P36/P35	Saída	Dado 8 bits
usb_wr	P21	Saída	WR# write strobe ativo baixo
usb_txe	P26	Entrada	TXE# FIFO não cheia

Pinos reservados — NÃO usar como I/O!

P29 = GND (terra)
P34 = VCCO_6 (alimentação Bank 6)

Atribuir estes pinos no UCF causa

```
Exceção pelo MAP/PAR do SE:  
NET "clk" LOC = "P53" | IOSTANDARD = LVCMOS33;  
NET "btn" LOC = "P56" | IOSTANDARD = LVCMOS33 | PULLDOWN;  
NET "usb_d<0>" LOC = "P31" | IOSTANDARD = LVCMOS33;  
NET "usb_wr" LOC = "P21" | IOSTANDARD = LVCMOS33;
```

Arquitetura VHDL: Dois Processos Paralelos

A lógica paralela do FPGA — não é código sequencial

Entradas

- P53** — **clk**
- game_proc**
 - SMT do jogo (7 estados)
 - Debug: reset + sincronizador
- P26** — **usb_tx**
 - LFSR (aleatoriedade)
 - Divisor de ms + BCD

- fifo_proc**
 - FIFO TX 245 (5 estados)
 - Aguarda uart_trigger
 - Envia byte a byte com WR#

Saídas — LEDs

- **led_reflexo** **P51** **D5**
- **led_status** **P50** **D6**
- **led_erro** **P47** **D7**

Saídas — FIFO USB

- **usb_d[7:0]** **P27-P36**

■ Paralelismo real no FPGA

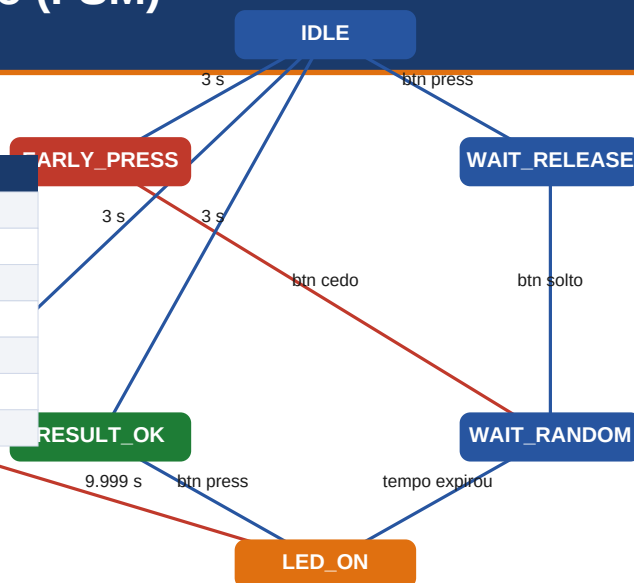
game_proc e fifo_proc executam ao mesmo tempo — não sequencial!
Software nunca consegue isso.

Máquina de Estados do Jogo (FSM)

7 estados · Transições por eventos de 1 ms

Comportamento dos LEDs por estado

Estado	D5	D6	D7	Significado
IDLE	—	D6	—	Aguardando
WAIT_RELEASE	—	—	—	Liberando botão
WAIT_RANDOM	—	—	—	Aguardando LED
LED_ON	D5	—	—	Medir reação
RESULT_OK	D5	D6	—	Resultado OK
EARLY_PRESS	—	—	D7	Pressionou cedo
TIMEOUT	—	—	D7	Sem reação



Aleatoriedade: LFSR de 16 bits

Por que o intervalo de espera é imprevisível

Um LFSR (Linear Feedback Shift Register) avança 40 milhões de vezes por segundo.

No instante exato do clique, capturamos 12 bits como valor aleatório.

LFSR de 16 bits — polinômio $x^{15}+x^{13}+x^{12}+x^{10}+1$:

15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

```
-- XOR dos taps: bits 15, 13, 12, 10
lfsr_feedback := lfsr(15) xor lfsr(13) xor lfsr(12) xor lfsr(10);
lfsr <= lfsr(14 downto 0) & lfsr_feedback; -- shift left

-- Ao pressionar o botão:
delay_target_ms <= 1000 + to_integer(lfsr(11 downto 0));
--                               ↑ min 1 s   ↑ 0 a 4095 ms = até ~5 s
```

Por que não um simples contador?

Problema com contador:

Sempre o mesmo intervalo →
jogador memoriza o padrão em 2 rodadas!

Solução com LFSR:

65.535 valores distintos antes de
repetir → padrão imprevisível

Propriedades do LFSR:

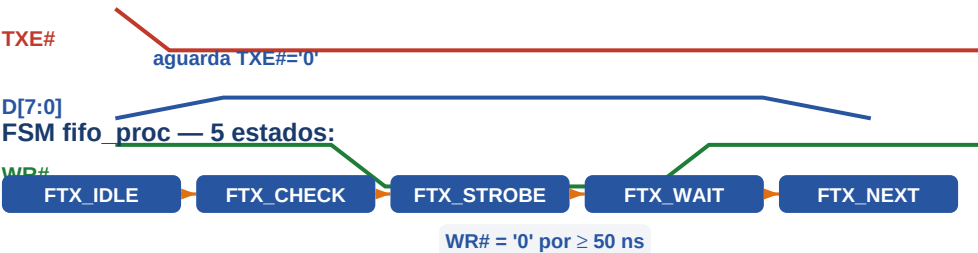
- Implementado em hardware (flip-flops + XOR)
- Zero custo: avança todo clock automaticamente
- Período: $2^{16} - 1 = 65.535$ estados únicos
- Intervalo final: 1000 ms a 5095 ms

Comunicação com o PC: FIFO 245 Assíncrono

Por que não UART — e como o protocolo funciona

O esquemático mostra que o Canal B do FT2232H expõe barramento de 8 bits (BDBU06-7) mais WR# e TXE#. Isso é Async FIFO 245 — não UART serial de 1 fio.

Protocolo de escrita (FPGA → FT2232H → PC):



Mensagens enviadas ao PC:

Evento	Mensagem ASCII	Bytes
Erro cometido:		
Tentamos usar 1 pino uart_tx (P28, P30, P29, P93) — zero bytes chegavam. Causa: Canal B é barramento paralelo, não UART serial.		
Monitor serial:		
/dev/ttyUSB1 (Linux) — Interface 01		
COM2 (Windows)		
Baudrate: qualquer (FT2232H ignora)		

Conversão BCD e Timing Violation

Como o FPGA transforma 342 em '0342' — e o problema de timing

Para transmitir RESULT_MS=0342, o FPGA precisa converter o inteiro 342 em ASCII.

Divisão não é suportada pelo VLSI para inteiros variáveis — usamos subtrações:

```
rms := reaction_time_ms; -- ex: 342
if rms >= 9000 then d3 := 9; rms := rms - 9000;
elsif rms >= 8000 then d3 := 8; rms := rms - 8000;
-- ... (10 ramos, do 9 ao 0)
-- Repete para centenas (d2), dezenas (d1), unidades (d0)

-- Converte para ASCII: '0' = 48
uart_msg(10) <= to_unsigned(d3 + 48, 8); -- '3'
uart_msg(11) <= to_unsigned(d2 + 48, 8); -- '4'
uart_msg(12) <= to_unsigned(d1 + 48, 8); -- '2'
uart_msg(13) <= to_unsigned(d0 + 48, 8); -- '0' ... espera
```

O problema de timing:

24 níveis de lógica combinacional

Atraso medido: 27,7 ns

Clock: 25 ns (40 MHz)

27,7 ns > 25 ns → FALHA de timing!

PAR reportou 16 timing errors

Solução: TIG no VLSI

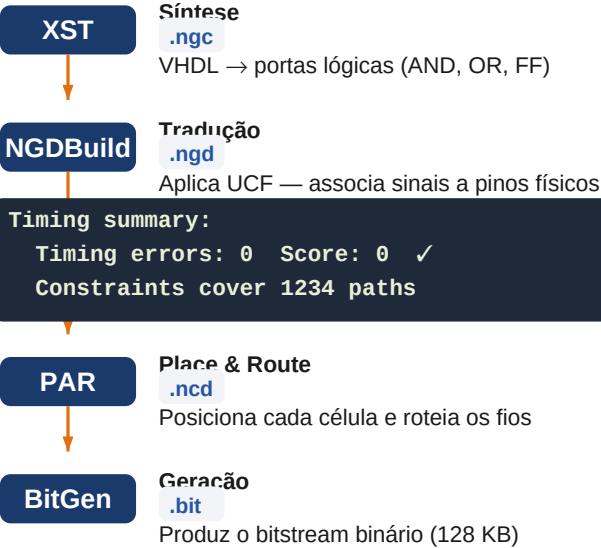
```
INST "reaction_time_ms_*" TNM = "TG_rms";
INST "uart_msg_*_*" TNM = "TG_uart_msg";
TIMESPEC "TS_bcd" =
    FROM "TG_rms" TO "TG_uart_msg" TIG;
```

Por que TIG é seguro aqui?

- Executa 1× por pressão de botão
- Não acontece em clocks consecutivos
- Valor já estabilizou antes da leitura
- Sem risco de metaestabilidade

Fluxo de Síntese ISE 14.7

Do código VHDL ao bitstream pronto para gravar



Resultado da síntese:

Recurso	Usado	Disp.	%
Slices	344	1.920	17,9%
LUTs	619	3.840	16,1%
Flip-Flops	120	3.840	3,1%
IOBs	14	97	14,4%
Block RAM	0	12	0%
Entradas de	0	—	■

Entradas do fluxo:

<code>reflex_game.vhd</code>	Código VHDL — lógica do jogo
<code>reflex_game.ucf</code>	Constraints de pinos e timing
<code>reflex_game.xst</code>	Opções do XST (device alvo)
<code>reflex_game.prj</code>	Lista de arquivos VHDL

Gravação do Firmware

SRAM (temporário) vs Platform Flash XCF01S (permanente)

Gravação na SRAM — temporário

```
# Via xc3sprog:  
xc3sprog -c ftdi reflex_game.bit
```

■ Resultado em ~3 s · Perde ao desligar

Gravação na Flash — permanente

```
# Via openOCD:  
openocd -f gravar.cfg -c init; pld load 0 reflex_game.bit; exit"
```

1. promgen

Converte .bit → .mcs

(formato da Platform Flash XCF01S) Script JTAG: apagar + gravar + verificar

2. iMPACT

Gera arquivo .svf

3. OpenOCD

Executa o .svf na placa

```
# Via xc3sprog (mais simples):
```

```
xc3sprog -c ftdi -p 1 reflex_game.bit:w:0:BIT
```

Cadeia JTAG da placa:

xc3s200

xcf01s

■ Após gravar na Flash:

Desligar USB · Desligar placa
Religar placa (sem PC, sem cabo)
XCF01S carrega bitstream automaticamente
D6 acende: jogo pronto! ■

Problemas Encontrados e Soluções

56 problemas catalogados — os mais críticos aqui

P40 FIFO 245, não UART

Problema:

Zero bytes chegavam em qualquer pino uart_tx testado (P28, P30, P93)

Solução:

Reescrever VHDL: barramento 8 bits + WR# + TXE#

P44 SVF iMPACT com 162 bytes

Problema:

Flash 'gravada' mas firmware não persistia após power cycle

Solução:

Checar rc iMPACT + validar tamanho mínimo do SVF (>1 KB)

P45 xcf04s → xcf01s

Problema:

IDCODE 0xd5044093 não bate com xcf04s esperado

Solução:

Cruzar IDCODE com BSDL do ISE: confirma xcf01s

P46 Cadeia JTAG invertida

Problema:

TDO sempre zero — FPGA e Flash em posições trocadas

Solução:

Flash=pos1 (TDO)
FPGA=pos2 (TDI)

P42 P29=GND, P34=VCCO

Problema:

PAR rejeitou pinos ao tentar usar como I/O

Solução:

Usar P27 e P36
(PAD report do ISE)

P43 Timing violation BCD

Problema:

27,7 ns > 25 ns no caminho reaction_time_ms → uart_msg

Solução:

TIG no UCF para esse path específico

Lição principal: leia o esquemático e o datasheet ANTES de escrever UCF e VHDL.

Testes e Evidências

Validação do sistema em bancada

Testes realizados:

■ Detecção JTAG

openocd reportou xc3s200=0x01414093 e xcf01s=0xd5044093

■ Compilação sem timing errors

PAR: Timing errors: 0 Score: 0

■ Gravação na RAM

pld load → return code 0 · D6 acendeu imediatamente

■ Flash permanente

Power cycle sem USB → D6 acendeu automaticamente

■ Jogo IDLE

D6 aceso ao ligar a placa

■ Sinal aleatório

D5 acendeu após intervalo variável (confirmado ~5 partidas)

■ EARLY (cedo demais)

Pressionar antes de D5 → D7 acendeu corretamente

■ TIMEOUT

Não reagir por 10 s → D7 acendeu

■ Dados FIFO no ttyUSB1

Porta aparece no sistema · Teste de bytes aguardando nova placa

```
$ openocd -f gravar.cfg ...  
Info : JTAG tap: xc3s200 tap/device found: 0x01414093  
Info : JTAG tap: platform_flash tap/device found: 0xd5044093  
Info : pld load 0 'reflex_game.bit'  
return code: 0  
$ # D6 acendeu – jogo rodando!
```

Conclusão e Aprendizados

Resultado final · O que levamos desta experiência

Resultado final:

- Jogo funcional gravado permanentemente na Flash XCF01S
- Carrega sem PC · 3 LEDs · botão · USB FIFO · resolução 1 ms
- 344 slices (17,9%) · 0 erros de timing

Sobre VHDL:

- VHDL descreve hardware paralelo — 2 processos rodam ao mesmo tempo
- FSMs são a estrutura central do design digital síncrono
- UCF é tão crítico quanto o VHDL — pino errado = sem saída de sinal
- TIG permite síntese limpa quando um caminho é logicamente seguro

Sobre processo:

- Ler datasheet + esquemático ANTES de codificar economiza horas
- Verificar IDCODE dos chips na cadeia JTAG antes de gerar SVF
- Validar tamanho do SVF evita falso-positivo de gravação

Ferramentas utilizadas:

Ferramenta	Uso
ISE 14.7	XST · NGDBuild · MAP · PAR · BitGen · iMPACT
OpenOCD	Gravação JTAG (pld load e svf)
xc3sprog	Alternativa simples para JTAG
promgen	Conversão .bit → .mcs (formato Flash)
Python/tkinter	Painel de controle com monitor serial

Recursos utilizados:

DS099	Spartan-3 FPGA Data Sheet
UG331	Spartan-3 User Guide
BSDL xcf01s	Identificação correta da Flash
PAD Report	Validação dos pinos do package TQ144